

Platform Architecture & Feature Documentation

Zero-dependency, multi-plane build provenance and cryptographic integrity platform for hackathon judging, pre-built code inspection, and forensic timeline analysis.

1. System Architecture: The Three Verification Planes

HACKPROOF organizes all verification across three independent planes to guarantee that local machine tampering, clock manipulation, or git history rewriting cannot forge a clean verdict.

1. SERVER PLANE

Independent Observer: Real-time GitHub webhooks (HMAC-SHA256), REST API synchronization, clock drift monitoring, and force-push history rewriting detection.

2. SYSTEM PLANE

Hardware Inspection: Local hardware devices, stealth index flags (`--skip-worktree`), `.git/info/exclude` overrides, and uncommitted source code on disk.

3. CLAIM PLANE

Git Repository History: Cryptographic signatures (GPG/SSH), build window boundaries (`--t0`), parent timestamp monotonicity, burst analysis, and `.gitignore` blame.

VERIFICATION PLANE	SOURCE OF TELEMETRY	FORENSIC DEFENSE PURPOSE
Claim Plane	Git commit graph, author dates, tree objects, <code>.gitignore</code> porcelain blame	Catches backdated commits, speedrun code dumping (<2s intervals), and late-added ignore rules hiding modules.
System Plane	Local working tree, git index metadata, excludes configs	Catches stealth index modifications (<code>git update-index --skip-worktree</code>) and untracked code on disk.
Server Plane	GitHub App webhooks, REST polling, SQLite Roster DB	Prevents clock tampering, detects force-pushes, flags unregistered laptops, and identifies long offline gaps (>4h).

2. Comprehensive Inventory of Implemented Features

1. Multi-Team Roster & Hardware Device Database

hackproof-roster

core/roster_db.py • SQLite Schema (hackproof.db)

- **Relational DB Model:** Manages teams, members, keys, and physical devices with foreign-key integrity.
- **Hardware Device Binding:** Enrolls physical laptop MAC addresses/hostnames at check-in to bind commits to authorized hardware.
- **Global Key Uniqueness:** Enforces global uniqueness on public keys to prevent cross-team identity sharing or collusion.
- **Enrollment Card Support:** Exports team cryptographic credentials and imports legacy JSON roster manifests.

2. Cryptographic Commit Signature Verification (§8.e) hackproof-analyze --roster hackproof.db

analyzers/gpg_check.py • GPG / OpenSSH Signatures

- **Signature Coverage:** Computes the exact percentage of commits cryptographically signed by registered team keys.
- **Unsigned Commit Detection:** Flags commits pushed without cryptographic signatures (unregistered laptops).
- **Outsider Key Detection:** Detects valid signatures signed by unknown keys not present on the team roster.
- **Identity Impersonation Detection:** Catches identity mismatches where Git author claims to be Person A, but signature is Person B.
- **Platform Fallback Keys:** Built-in support for platform-signed commits (e.g., GitHub Web-flow automated merge keys).

3. File Concealment & Pre-Built Code Staging (§8.f)

analyzers/gitignore_check.py

System & Claim Plane File Verification

- **Tracked Source Ratio:** Compares files tracked in Git against files on disk to uncover massive hidden source trees.
- **Index Bit Tampering:** Identifies stealth --skip-worktree and --assume-unchanged index flags.
- **Local Exclude Overrides:** Audits .git/info/exclude and core.excludesFile overrides that never leave the laptop.

4. Forensic .gitignore Blame View (§8.f) & Main Window Table hackproof-analyze • Section [2]

Porcelain Line Attribution & Active Concealment

- **Porcelain Blame Parser:** Uses git blame --line-porcelain for machine-attributed line numbers, authors, and timestamps.
- **Timing Classification:** Identifies repository root commit epoch; computes [INIT] vs [LATE +X.Xh] offset for every line.
- **Active Concealment Warning:** Uses git check-ignore -v to flag 🚫 CONCEALED when late rules hide uncommitted source files.
- **Direct Main Window Integration:** Formatted table renders natively inside the Executive Summary Card by default.

5. Build Window, Interval & Monotonicity Forensics (§8.d / §8.i)

analyzers/claim_timestamp_check.py

Timestamp Integrity & Typist Dynamics

- **Kickoff Window Verification (`--t0`)**: Flags any commits created before the official hackathon start time.
- **Parent Monotonicity**: Guarantees chronological order; detects backdated parent commits, rebases, and clock tampering.
- **Speedrun / Burst Detection**: Flags superhuman commit quantization (< 2s intervals) indicating scripted code injection.
- **Timezone Consistency**: Validates timezone offsets across commits to detect offshore proxy developers.

6. Stage C Server Plane: Webhook Listener & REST Polling

hackproof-server • hackproof-poll

server/webhook_server.py • server/github_service.py

- **HMAC-SHA256 Webhook Server**: Standard library HTTP server with constant-time signature verification on `X-Hub-Signature-256`.
- **Dual-Timestamp Drift Tracking**: Computes drift between GitHub's `created_at` and server receipt time to detect clock skew.
- **Force-Push Detection**: Records and flags any non-fast-forward updates that overwrite hackathon history.
- **Push Gap Forensics**: Flags participants working offline for >4 hours followed by massive code dumps.
- **REST Polling Client**: Synchronizes repository creation date, branch refs, Actions runs, and GitHub signature verification.

7. Automated Remote GitHub Repo Auditing & Auto-Clone Engine

hackproof-analyze
<owner/repo>

Universal Audit Target Support

- **Direct Remote Auditing**: Audits any public/private GitHub repository without manual clone steps.
- **Full History Preservation**: Clones full history (never shallow) into an isolated sandbox to ensure complete telemetry.
- **Evaluability Context**: Automatically handles clone boundaries, reporting machine-local checks as `n/a` instead of false clean.
- **Automatic Teardown**: Removes temporary sandbox directories upon completion.

3. Command Reference

```
# 1. Audit any remote GitHub repository directly:
hackproof-analyze octocat/Hello-World

# 2. Audit local repository with roster and event start timestamp:
hackproof-analyze . --roster hackproof.db --team-id team-pritim --t0 2026-09-13T01:00:00+05:30

# 3. Export complete machine-readable audit report to JSON:
hackproof-analyze . --format json --out audit-results.json

# 4. Start Stage C GitHub Webhook Server:
hackproof-server --host 0.0.0.0 --port 8080 --db hackproof.db --secret "YOUR_SECRET"

# 5. Poll remote GitHub repository metadata into SQLite:
hackproof-poll owner/repo --db hackproof.db

# 6. Manage multi-team roster database:
hackproof-roster list
hackproof-roster team create --team-id team-alpha --name "Team Alpha"
```

4. Verification & Test Suite Matrix

146 Passing Tests: Zero external dependencies. All modules run using standard library Python + Git CLI.

TEST SUITE	FILE LOCATION	COVERAGE SCOPE
Engine & CLI	tests/test_engine.py	Isolation, shallow clone detection, summary card, CLI arguments
GPG & Key Binding	tests/test_gpg_check.py	Signature coverage, unsigned commits, outsider keys, platform keyrings
Gitignore & Blame	tests/test_gitignore_check.py	Porcelain blame, active concealment, skip-worktree, tracked ratios
Claim Timestamps	tests/test_claim_timestamp_check.py	Monotonicity, build window (--t0), interval bursts, timezone offset
Server Plane	tests/test_server_plane.py	HMAC verification, push event parsing, force-push detection, REST client
Roster DB	tests/test_roster_db.py	Relational schema, key uniqueness, hardware device binding
Remote Clone	tests/test_clone_analysis.py	Clone evaluability, credential scrubbing, system plane reporting